HandsOn 5

Task 1: Create the Collection and Insert Documents
Goal: Set up the MongoDB collection and insert realistic feedback documents.
Steps:
60. In MongoDB Compass or mongosh, create a database named college_nosql.
61. Create a collection named feedback.
62. Insert at least 10 feedback documents following the schema above. Vary ratings (1–5), tags, and
semesters. Include at least 3 documents for CS101 and 2 for CS102.
63. Insert one document that intentionally omits the attachments field — MongoDB's schema-less design
allows this.
64. Verify the inserts using db.feedback.countDocuments().

>_MONGOSH

```
> use  college_nosq
< switched to db college_nosq
> db.feedback.insertMany([
    { student_id: 1, course_code: 'CS101', semester: '2022-ODD', rating: 5,
      comments: 'Excellent teaching. Would recommend.',
      tags: ['challenging', 'well-structured', 'good-examples'],
      submitted_at: ISODate('2022-11-30T10:15:00Z'),
      attachments: [{ filename: 'notes.pdf', size_kb: 240 }] },

    { student_id: 2, course_code: 'CS101', semester: '2022-ODD', rating: 4,
      comments: 'Good pace, clear examples.',
      tags: ['well-structured', 'good-examples'],
      submitted_at: ISODate('2022-11-28T09:00:00Z'),
      attachments: [{ filename: 'slides.pdf', size_kb: 180 }] },

    { student_id: 3, course_code: 'CS101', semester: '2021-EVEN', rating: 2,
      comments: 'Too fast, hard to follow.',
      tags: ['challenging'],
      submitted_at: ISODate('2021-05-10T11:00:00Z'),
      attachments: [{ filename: 'notes.pdf', size_kb: 90 }] },

    { student_id: 4, course_code: 'CS102', semester: '2022-ODD', rating: 5,
      comments: 'Loved the assignments.',
      tags: ['well-structured', 'good-examples'],
      submitted_at: ISODate('2022-12-01T14:20:00Z'),
      attachments: [{ filename: 'project.pdf', size_kb: 300 }] },
```

>_MONGOSH

```
{ student_id: 5, course_code: 'CS102', semester: '2022-ODD', rating: 3,
  comments: 'Average, could be better organized.',
  tags: ['challenging'],
  submitted_at: ISODate('2022-11-25T16:45:00Z'),
  attachments: [{ filename: 'notes.pdf', size_kb: 150 }] },

{ student_id: 6, course_code: 'CS103', semester: '2022-ODD', rating: 1,
  comments: 'Needs improvement.',
  tags: ['challenging'],
  submitted_at: ISODate('2022-11-20T08:30:00Z'),
  attachments: [{ filename: 'notes.pdf', size_kb: 60 }] },

{ student_id: 7, course_code: 'CS103', semester: '2021-EVEN', rating: 4,
  comments: 'Solid course overall.',
  tags: ['well-structured'],
  submitted_at: ISODate('2021-04-15T10:10:00Z'),
  attachments: [{ filename: 'notes.pdf', size_kb: 200 }] },

{ student_id: 8, course_code: 'CS104', semester: '2022-ODD', rating: 5,
  comments: 'Best course this semester.',
  tags: ['well-structured', 'good-examples', 'challenging'],
  submitted_at: ISODate('2022-11-29T13:00:00Z'),
  attachments: [{ filename: 'notes.pdf', size_kb: 220 }] },
```

```
>_MONGOSH

    { student_id: 9, course_code: 'CS104', semester: '2022-ODD', rating: 2,
      comments: 'Content unclear at times.',
      tags: ['challenging'],
      submitted_at: ISODate('2022-11-27T15:30:00Z'),
      attachments: [{ filename: 'notes.pdf', size_kb: 100 }] },


    { student_id: 10, course_code: 'CS101', semester: '2022-ODD', rating: 4,
      comments: 'Great real-world examples.',
      tags: ['good-examples', 'well-structured'],
      submitted_at: ISODate('2022-11-30T17:00:00Z') }
  ])
  db.feedback.countDocuments()
< 10
```

Task 2: CRUD Operations
Goal: Practise all four CRUD operations on the feedback collection.
Steps:
65. READ: Find all feedback documents where rating is 5.
66. READ: Find feedback for course CS101 where the tags array contains 'challenging'. Use $elemMatch
or a simple array value query.
67. READ: Retrieve only the student_id, course_code, and rating fields (projection) for all documents —
exclude _id.
68. UPDATE: For all feedback documents with rating < 3, add a field needs_review: true using
updateMany and $set.
69. UPDATE: Push a new tag 'reviewed' into the tags array of all documents where needs_review is true,
using $push.
70. DELETE: Delete all feedback documents where the semester is '2021-EVEN'

```
> db.feedback.find({ rating: 5 })
  db.feedback.find({ course_code: 'CS101', tags: 'challenging' })
  db.feedback.find({}, { student_id: 1, course_code: 1, rating: 1, _id: 0 })

  db.feedback.updateMany(
    { rating: { $lt: 3 } },
    { $set: { needs_review: true } }
  )

  // UPDATE
  db.feedback.updateMany(
    { needs_review: true },
    { $push: { tags: 'reviewed' } }
  )

  // DELETE - remove old semester feedback
  db.feedback.deleteMany({ semester: '2021-EVEN' })
< {
    acknowledged: true,
    deletedCount: 2
  }
```

Task 3: Aggregation Pipeline
Goal: Build a multi-stage aggregation pipeline to generate analytical reports.
Steps:
71. Write a pipeline that: (Stage 1) filters to semester '2022-ODD'; (Stage 2) groups by course_code
calculating average rating and total feedback count; (Stage 3) sorts by average rating descending.
72. Extend the pipeline with a $project stage to rename avg_rating to average_rating and round it to 1
decimal place using $round.
73. Write a pipeline that uses $unwind on the tags array, then $group by tag to count how many times
each tag appears. Sort by count descending — a tag frequency leaderboard.

74. Add an index on course_code and verify its usage with db.feedback.find({course_code:'CS101'}).explain('executionStats') — confirm the stage shows
IXSCAN not COLLSCAN

```
> db.feedback.aggregate([
    { $match: { semester: '2022-ODD' } },
    { $group: {
        _id: '$course_code',
        avg_rating: { $avg: '$rating' },
        total_feedback: { $sum: 1 }
    }},
    { $sort: { avg_rating: -1 } }
  ])

  db.feedback.aggregate([
    { $match: { semester: '2022-ODD' } },
    { $group: {
        _id: '$course_code',
        avg_rating: { $avg: '$rating' },
        total_feedback: { $sum: 1 }
    }},
    { $sort: { avg_rating: -1 } },
    { $project: {
        _id: 0,
        course_code: '$_id',
        average_rating: { $round: ['$avg_rating', 1] },
        total_feedback: 1
    }}
  ])
  db.feedback.aggregate([
    { $unwind: '$tags' },
    { $group: { _id: '$tags', count: { $sum: 1 } } },
    { $sort: { count: -1 } }
  ])
  db.feedback.createIndex({ course_code: 1 })
  db.feedback.find({ course_code: 'CS101' }).explain('executionStats')
```

```
  indexFilterSet: false,
  queryHash: '83CE04A5',
  planCacheShapeHash: '83CE04A5',
  planCacheKey: 'E4E1D11D',
  optimizationTimeMillis: 0,
  maxIndexedOrSolutionsReached: false,
  maxIndexedAndSolutionsReached: false,
  maxScansToExplodeReached: false,
  prunedSimilarIndexes: false,
  winningPlan: {
    isCached: false,
    stage: 'FETCH',
    nss: 'college_nosq.feedback',
    inputStage: {
      stage: 'IXSCAN',
      nss: 'college_nosq.feedback',
      keyPattern: {
        course_code: 1
      },
      indexName: 'course_code_1',
      isMultiKey: false,
      multiKeyPaths: {
        course_code: []
      },
```